FinMarket-Enterprise

Financial Advisory Marketing and Client Acquisition Platform

Team project guide | Modern Web Technologies | Five-member group

Updated guide — all three official project phases clearly identified.

1. What are we building?

FinMarket-Enterprise is a web application that helps financial advisors attract potential clients and reduce repetitive marketing work. Advisors can manage prospects (leads), create campaigns, prepare emails and Facebook Page content, arrange follow-ups, and monitor results.

Example: An advisor creates a retirement-planning campaign, selects leads who have consented to marketing, queues an email, receives a consultation request, and sees the lead status and campaign figures update. This complete process is the long-term goal—not a Phase I requirement.

2. What does the professor actually require?

The course specification defines THREE phases. The statements below reproduce the deliverable categories in Section 7; the tasks later in this document are our suggested implementation plan.

Phase I — Backend Foundation (current submission)

- Business case definition
- Backend project setup
- Core API routes
- Initial data models
- GitHub repository initialization

Phase II — Database and Security Integration

- MongoDB integration using Mongoose
- Authentication and authorization implementation
- Role-based access enforcement
- Secure data handling

Phase III — Frontend Integration and Deployment

- React frontend integration
- Full-stack connectivity
- Deployment to a hosting platform
- Production environment configuration

IMPORTANT: Phase I does not explicitly require a finished React frontend, MongoDB connection, JWT authentication, live Gmail delivery, or Facebook integration. Those are later-phase or proposed project features. The professor has not stated in the specification whether Phase I routes must already process realistic test data; the team recommends doing so to make the backend demonstrable.

Final project minimums (not all due in Phase I)

Requirement	Final project expectation
Backend	Node.js + Express.js

Database	MongoDB + Mongoose, at least 6 collections
Frontend	React
Security	JWT, password hashing, backend-enforced roles
API	At least 10 RESTful endpoints, at least 3 protected
Professional practice	Postman, one private GitHub repository, at least 5 meaningful commits per student
Final delivery	README, source code, deployment, group presentation

3. Phase I: exactly what we need to prepare

A. Business case definition README.md

Explain the problem, the users, the proposed solution, and the main workflows. Example problem: financial advisors manually maintain prospect lists, prepare marketing messages, and remember follow-ups; important opportunities may be missed.

Our proposed roles: Administrator (manages users and permissions), Financial Advisor (manages their profile, leads and campaigns), and Marketing Specialist (prepares campaigns and delivery activity under permitted access).

Write 2–3 workflows: (1) advisor creates campaign; (2) lead moves from New to Contacted to Consultation Requested to Converted; (3) a campaign can queue a message only for a contact with marketing consent.

B. Backend project setup

Create backend/ as a Node.js + Express.js ES-module project. Configure JSON request parsing, CORS, environment variables, versioned routes, centralized error handling, and GET /api/v1/health. Split app initialization from server startup.

C. Core API routes

Create the route files for all five modules. Recommended: implement basic request validation and temporary in-memory records so routes can be tested before MongoDB is connected. These are team recommendations, not extra official Phase I requirements.

D. Initial data models

Describe what each collection will store and how records relate. Define initial Mongoose schema files if possible, even though connecting MongoDB is a Phase II task. At least six MongoDB collections are required in the FINAL project; we propose eight below.

E. GitHub repository initialization

Create one PRIVATE repository named finmarket-enterprise, invite all five group members, add a .gitignore and README, and agree on separate feature branches. Everyone must contribute genuine code and be able to explain their work.

Phase I team checklist

- Document business problem, goals, user roles and example workflows.
- Initialize an executable Express backend and health endpoint.
- Create all five modules and their initial routes.
- Define and document initial models and relationships.
- Initialize private GitHub repo, invite members, add README and commit history.
- Recommended: test each module through Postman and include sample requests.

4. How the final application should work

Feature	What it does
Identity & advisor profiles	Register users, sign in, edit specialties, configure consultation types.
Lead management (CRM)	Store prospective clients, filter them, record consent and change lead status.
Campaigns & templates	Create campaigns for retirement planning, webinars or newsletters; save reusable content.
Marketing delivery	Queue and track email campaigns; connect authorized Gmail and supported Facebook Page actions.
Automation & analytics	Trigger welcome/follow-up actions and summarize leads, delivery results and conversions.

Safety and practicality: Do not send unwanted bulk email. Use consenting test recipients, unsubscribe handling, rate limits and secure credential storage. Gmail sending limits apply. Meta/Facebook Page features depend on access permissions and API approvals; a mock provider is the fallback for demonstrations.

5. Five team members: clear independent responsibilities

Each student owns ONE complete feature from routes and initial models in Phase I through database integration in Phase II and their React screens in Phase III. This division aims for similar responsibility, but the actual effort may vary. Modules should work with sample IDs and mock services so nobody needs to wait for another student.

Member 1 — Users & Advisors

Owns: backend/src/modules/identity/ and related tests/frontend feature pages. Models: User and AdvisorProfile.

PHASE I — Registration/login routes and advisor listing/profile endpoints; define role rules.

PHASE II — Implement hashed passwords, JWT authentication and authorization middleware; persist users and profiles.

PHASE III — Build login, registration and advisor profile/directory screens.

Done when: another teammate can test this module's endpoint(s), view an example result, and understand the member's README explanation.

Member 2 — Lead Management

Owns: backend/src/modules/leads/ and related tests/frontend feature pages. Models: Lead.

PHASE I — Create/list/update/filter lead routes; validate email, consent and lifecycle statuses.

PHASE II — Persist leads, prevent unauthorized access and duplicate contacts as appropriate; enforce status changes.

PHASE III — Build lead directory, details, editing and filtering screens.

Done when: another teammate can test this module's endpoint(s), view an example result, and understand the member's README explanation.

Member 3 — Marketing Campaigns

Owns: backend/src/modules/campaigns/ and related tests/frontend feature pages. Models: Campaign and Template.

PHASE I — Create/list/update campaigns and retrieve templates; define strategies and campaign statuses.

PHASE II — Persist campaigns/templates, implement target audience selection and campaign status rules.

PHASE III — Build campaign list/editor and template screens.

Done when: another teammate can test this module's endpoint(s), view an example result, and understand the member's README explanation.

Member 4 — Email & Facebook Delivery

Owns: backend/src/modules/delivery/ and related tests/frontend feature pages. Models: DeliveryJob.

PHASE I — Create/list delivery jobs, mock provider responses, design email/social delivery contracts.

PHASE II — Implement permitted Gmail SMTP integration, queue/rate limits, delivery tracking; attempt allowed Meta Page operation if approved. Meta API integration can be used for publishing text posts to an authorized Facebook Page.

PHASE III — Build delivery history and channel-configuration screens.

Done when: another teammate can test this module's endpoint(s), view an example result, and understand the member's README explanation.

Member 5 — Automation & Analytics

Owns: backend/src/modules/automation/ and related tests/frontend feature pages. Models: AutomationRule and ActivityEvent.

PHASE I — Create/list automation rules and basic analytics summary, simulate a sample event.

PHASE II — Persist rules/events, process triggers, avoid duplicate execution and schedule follow-ups.

PHASE III — Build automation rule editor and dashboard.

Done when: another teammate can test this module's endpoint(s), view an example result, and understand the member's README explanation.

Member	Directory	Main responsibility
1	modules/identity/	Users, authentication, advisor profiles
2	modules/leads/	Lead management and CRM
3	modules/campaigns/	Campaigns and marketing templates
4	modules/delivery/	Gmail and Facebook Page publishing
5	modules/automation/	Marketing automation and analytics

6. Data models: simple starting point

A data model is the plan for information saved in a database. For example, a Lead may have name, email, interest, status, marketingConsent and advisorId. The advisorId relates that lead to its financial advisor.

Proposed collection	Example information
users	name, email, passwordHash, role
advisorProfiles	userId, specialties, consultationTypes
leads	advisorId, name, email, status, marketingConsent
campaigns	advisorId, title, audience, status
templates	name, subject, body, channel
deliveryJobs	campaignId, leadId, channel, deliveryStatus
automationRules	ownerId, trigger, action, enabled
activityEvents	entityId, eventType, occurredAt

These eight collections are our team design; the professor specifies a minimum of six, not these exact names. Final schemas should define required fields, allowed values, defaults, relationships and useful indexes.

7. Suggested API endpoints for the backend foundation

API = a route another program can call. GET reads, POST creates, PATCH updates. All endpoints below are examples chosen by our team; they are not exact routes prescribed by the professor.

Owner	Route	Purpose
1	POST /api/v1/auth/register	Register user
1	POST /api/v1/auth/login	Sign in
1	GET /api/v1/advisors	List advisors
2	POST /api/v1/leads	Add lead
2	GET /api/v1/leads	Find leads
2	PATCH /api/v1/leads/:id	Update lead
3	POST /api/v1/campaigns	Add campaign
3	GET /api/v1/campaigns	List campaigns
3	PATCH /api/v1/campaigns/:id	Update campaign
4	POST /api/v1/deliveries	Queue delivery (mock in Phase I)
4	GET /api/v1/deliveries	View delivery history
5	POST /api/v1/automations	Create rule
5	GET /api/v1/automations	List rules
5	GET /api/v1/analytics/summary	Summary counts

Also implement GET /api/v1/health to verify the server. Example: invalid required data should produce a clear 400 response; a missing record should return 404. Do not treat mocked Phase I login as real authentication.

8. Architecture explained in plain language

Request flow: Route → Controller → Service → Repository → Mongoose model → MongoDB (database connection comes in Phase II).

- Route: receives an HTTP URL such as GET /api/v1/leads.
- Controller: reads the request and prepares the response.
- Service: enforces business rules, such as requiring consent before marketing.
- Repository: reads or writes information; Phase I can use mock/in-memory storage, Phase II uses database queries.
- Model: defines database fields and validation; MongoDB holds saved records.

Each module should contain its own routes, controller, service, repository and models folder. Shared files such as app.js and error middleware should be set up once and changed through coordinated pull requests.

9. How to work without blocking other members

1. Agree on endpoint paths and sample JSON before coding.
2. Use advisorId, campaignId and leadId as references rather than importing another member's business logic.
3. Test with fake example records. Do not wait for real user registration or email delivery.
4. Work in separate feature branches: feature/identity, feature/leads, feature/campaigns, feature/delivery, feature/automation.
5. Make small meaningful commits and request code reviews before merging into main.

Example: Member 5 can simulate an event named LEAD_CREATED with a sample leadId. Later, Member 2 makes the actual leads module emit the same event. Members can build independently because they agreed on the event shape first.

10. What we submit and how to check readiness

For Phase I, the course lists the five foundation items in Section 7. The overall submission section additionally specifies a private GitHub repository, application source code, README and a minimum of five meaningful commits per student. It does NOT explicitly give a separate Phase I rubric or say every final requirement must already be implemented.

Ready-to-share Phase I handoff

- Private GitHub repository accessible to all five teammates.
- README: business problem, users, project goal, setup instructions, module owners and future phases.
- Express backend starts and health endpoint responds.
- Each feature has initial API routes and models with documented responsibilities.
- Recommended: each student supplies working mock examples and Postman requests.
- Everyone has meaningful contributions; review the course-wide five-commit requirement.

11. What happens after Phase I?

PHASE II: Connect MongoDB using Mongoose, replace mock storage with real persistence, add secure JWT login, password hashing, role-based permission checks and secure data handling. Test blocked actions and bad input.

PHASE III: Build React screens for each of the five feature areas, connect frontend actions to Express APIs, integrate automated delivery and follow-ups at an achievable scope, deploy, configure the production environment and present the completed application.

Remember: the final course project is evaluated on backend/API design, database/data integrity, security, frontend integration, deployment, and professional contribution. Keep evidence of tests, technical decisions and individual commits.

12. Short glossary

Term	Plain-language meaning
Backend	The server code that processes requests and applies rules.
API endpoint	URL and HTTP method the frontend or Postman calls.
CRUD	Create, Read, Update, Delete information.
Mongoose model	A JavaScript schema for MongoDB data.
JWT	A signed token used to identify authenticated requests.
Mock data	Temporary example records used before a real service is ready.
Lead	A person who may become a client.
Campaign	A planned marketing effort aimed at a group.
Automation rule	An instruction such as “when a lead is created, queue a welcome message.”

Source: Course Project Specification Document: Modern Web Technologies, Sections 4–10 and 12. FinMarket-Enterprise feature scope, ownership, endpoints, and recommended Phase I mock implementations are proposed team decisions, not additional professor-issued requirements.